

Machine Learning-Enhanced Spectral Sensing for Advanced Color Discrimination Under Varying Ambience

Aiden Aysun, Brody Aysun, Colin Aysun,

Oviedo High School

Abstract

In this project, we build a spectrometer that can distinguish colors under different background lighting conditions. In doing so, we use a machine learning algorithm to train the spectrometer to precisely detect colors. Without machine learning, color detection is far less precise under varying illuminance. Our approach improves conventional methods that rely solely on reference-based calibration algorithms, which can be unreliable in uncontrolled environments. Specifically, the machine learning model that we implement with raw spectral data from a low-cost 10-channel AS7341 sensor provides more accuracy in color detection under different lighting conditions by learning ambience-independent patterns from sensor outputs. We begin by obtaining cosine similarity scores from our baseline tests without machine learning. We then use a softmax algorithm with temperature to convert the cosine similarity scores to percentages so that we can compare them with the probability scores that we obtain from our tests with machine learning. We find that the probability scores are considerably higher with machine learning.

Index Terms

Machine-Learning, spectrometer, color detection, ambience, cosine similarity, softmax.

Roles: Colin Aysun: Project Manager: This includes assigning tasks for the day, ordering parts, and managing software and hardware. Brody Aysun: Software Lead: Specifies in creating a sound project structure including downloading IDEs, reading color channels, and analyzing input. Aiden Aysun: Hardware / CAD Lead: Creates any 3D-printable parts and works on assembling hardware such as soldering, spectral to arduino connection, ensuring we have all the necessary parts, etc. General Note: All of us share the workload of this project almost evenly where we are all programming, designing, and managing. The roles above should be taken lightly, with almost every task in this project being completed by all of us in a shared, collaborative manner, with the roles only signifying slight authority. To make sure we all had the latest version of code we managed our programming files with version control via GitHub: <https://github.com/caysun/Spectral-Robotics>

I. INTRODUCTION

Precise color and spectral detection is critical in areas such as agriculture quality control, colorimetric assays, star identification, and traffic light detection for autonomous vehicles. In these areas, accuracy of detection under different background lighting conditions is highly important (e.g., night and day for autonomous vehicles).

The conventional methods for color detection under varying conditions have mostly used RGB, camera-based, or pre-calibrated detection. There have been well-known failures of these methods due to shortcomings related to equipment and image processing. For example, Gené-Mola et al. (2020) use an RGB-D sensor to detect ripeness in apple orchards and find that the sensor resolution and precision decrease significantly under middle to high ambient illuminance, causing a misclassification of fruit characteristics and a decline in apple yield.

Our approach provides an alternative by implementing a machine learning (ML) model that is trained with spectral sensor data to provide a more versatile color detection system under different lighting conditions. Compared to a baseline cosine-similarity test with pre-calibrated values, our findings suggest that color detection accuracy is improved when paired with a Random Forest classification model. Specifically, with this methodology we were able to discern nuanced differences and patterns between the different paint swatches, independent of the clear value. Obtaining four separate readings for five colors under four different degrees of illuminance, we find that there is roughly an 87% reduction in average squared error for the ML approach relative to the conventional cosine-similarity approach with pre-calibrated values. These results indicate that machine learning-enhanced spectral sensing is a promising pathway to more precise object and color detection under varying ambience in large-scale applications such as industrial robotics, agricultural quality control, and medical sampling.

II. LITERATURE REVIEW

Our work is related to the broad literature on spectral sensing and color detection. Much of this research employs advanced sensors and sophisticated machine-learning algorithms. For instance, Flachot et al. [3] develop deep neural networks modeled on human perception using complex, high-dimensional input, while Erba et al. [2] adapt classical RGB color constancy algorithms to multispectral inputs and refine them to reduce angular error in illuminant estimation. Similarly, Hamidisepehr and McDonald [5] examine the accuracy of portable spectrometers in ambient light, comparing different classification and lighting-compensation methods for agricultural applications. Our work differs in that we directly compare Random Forest classification with a non-machine-learning baseline for color detection under multiple controlled illuminations.

Khanal et al. [6] demonstrate how a machine-learning-assisted, camera-based system can improve the analysis of colorimetric assays, whereas we train our model using data from a multi-channeled spectrometer rather than an imaging setup. Contreras et al. [1] explore explainable artificial intelligence (XAI) for spectroscopy; in contrast, we focus on a single, well-defined algorithm—Random Forest—to enhance classification accuracy rather than model interpretability. Drawing inspiration from Koskinen et al. [7], who compare two input approaches for training a regressor to estimate scene white points, we likewise evaluate a machine-learning classifier against a classical cosine-similarity baseline. However, our goal is distinct: predicting

discrete color labels under variable lighting using a low-cost spectral sensor (AS7341) rather than estimating continuous white-point values with a commercial spectrometer.

Unlike prior studies, our task requires less preprocessed, lower-dimensional input, allowing a simpler Random Forest classifier to achieve high accuracy. Moreover, we bypass illuminant correction entirely, comparing cosine similarity of raw spectra and learning illumination-invariant mappings directly with the Random Forest model. In sum, our findings show that a low-budget sensor paired with a simple machine-learning algorithm can substantially improve color detection performance.

III. INSTRUMENTS, METHODOLOGY, AND FINDINGS

In this section, we describe the methodology and materials that we use for color detection in different environments. We outline the machine-learning algorithm that is used to train the readings. Finally, we discuss our findings.

A. Instruments

To conduct our experiments, we used an AS7341 spectral sensor and an Arduino NANO V3.0 Atmega328P board. We connect these two components using female-female jumper wires. On the spectral sensor, we collect data from 10 channels. These channels range between 415nm-680nm, including a clear sensor.¹ It is important to note that this 10-channel sensor is a recent technology that provides more detailed data readings than conventional RGB or HSV sensors. In particular, the clear channel provides a reading proportional to the overall light intensity. This reading is critical for our analysis, as it allows us to account for varying ambiance. The 5 header pins of the sensor were unattached and had to be soldered to their corresponding channels (VIN, GND, SCL, SDA, GPIO). VIN denotes the input voltage (5V), which powers the sensor. GND denotes ground. SCL is the Serial Clock Line that syncs data readings. SDA is the Serial Data Line; the wire transmits data from the sensor to the board. GPIO is an output channel that contains an internal white LED that illuminates the surface.²

The communication between our sensor and the Arduino board is established by I^2C . The board sensor system, in turn, is connected to a laptop via a USB cable. We used the Arduino IDE to read the 10 data channels. In doing so, the DFRobots library allowed us to configure the spectral sensor and trigger the LED during readings. The readings were then printed on a serial monitor in json format so that they could be processed in Visual Studio Code.

When using the built-in white LED from the AS7341 spectral sensor, we placed the sensor inside a custom 3D-printed shroud—designed in Fusion360 and sliced with PrusaSlicer—to block out all ambient light. For the other three lighting conditions (WHITE NORMAL, WHITE DIM, and YELLOW), we shined white and yellow LED light onto the color samples. To evaluate the effectiveness of machine learning for color detection, we trained a Random Forest model using the scikit-learn library.

¹F1: 415nm, F2: 445nm, F3: 480nm, F4: 515nm, F5: 555nm, F6: 590nm, F7: 630nm, F8: 680nm, Clear, Near Infrared.

²For the LED, we initially used a phone flashlight but this cast a shadow on the sample. After switching to the internal LED, our readings were far more accurate.

B. Methodology

We used our spectrometer to record measurements for five colors—red, green, blue, yellow, and orange³

We began gathering the benchmark values for the cosine similarity, one for each lighting condition and color pair (20 total). For each pair, we collected 10 raw channel readings from the sensors. We then divided each channel from F1-F8 by the given clear value and averaged the 10 readings. To minimize measurement error, we calibrated the averaged readings with the formula below:

$$\text{calibrated} = \frac{\text{sample} - \text{darkRef}}{\text{whiteRef} - \text{darkRef}} \quad (1)$$

where sample is the averaged readings divided by the clear value, darkRef is the averaged readings divided by the clear value when covering the sensor, and whiteRef is the averaged readings divided by the clear value when the sensor is directed at white Teflon (PTFE) tape. Teflon tape is effective because it reflects bright white light diffusely.

Next, we trained our machine learning model. We obtained input data with the same methodology that we used for cosine similarity. We saved and formatted the input values in a JSON file. Each of the 80 samples in the file has 9 features: 8 normalized channel readings (F1-F8), constituting our X variables and the color label that is our Y variable. We use this input data to train and test our Random Forest (RF) classifier. In specifying the RF model, we followed standard practice and created 100 decision trees with pure leaf nodes and implemented an 80/20 training-test split.

To test the accuracy of the cosine similarity model and the machine learning model, we use the same methodology. For each color and lighting condition pair, we again collect 4 raw channel readings from the sensors. After each reading, the program creates a sample with the 8 features defined above.

For the cosine similarity model, we compare these outputs with the 20 calibrated samples using the formula below:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2)$$

where $\mathbf{A}$ is the vector representing channels F1-F8 for the calibrated readings and $\mathbf{B}$ is the vector representing the normalized F1-F8 readings from the test data.

We then add the cosine similarity values of the benchmark channels that have the same color. To compare the accuracy of the two detection methods, we convert the cosine similarity values to percentages. To do this, we softmax the data as described by the following formula:

$$p_i = \frac{e^{s_i/T}}{\sum_j e^{s_j/T}} \quad (3)$$

where p_i is the calculated percentage, s_i is the current cosine similarity value we convert, T is the temperature⁴, and the

³The color samples were obtained from Home Depot paint swatches.

⁴Temperature is a scaling factor: values of $T > 1$ will produce a flatter (more uniform) distribution while values of $T < 1$ produce a sharper (more distinct) distribution

denominator is the sum of the exponentiated (and temperature-scaled) cosine similarity scores of all colors, ensuring that the calculated percentages sum to one.

To make the comparison more accurate, we also have to tune the temperature constant T . For this tuning, we use the negative log-likelihood function given by the following equation:⁵

$$\text{NLL}(T) = -\frac{1}{N} \sum_{i=1}^N \log(p_{\text{true class},i}(T)) \quad (4)$$

where T is the temperature value, $p_{\text{true class},i}(T)$ is the calculated probability of the true color label, and N is the number of samples. In order to scale the optimal temperature, we aimed to minimize this function. Because we only have one parameter, a simple grid search will be efficient. Since the function is not differentiable, we use graphical analysis to estimate its minimum value. Specifically, we graphed $\text{NLL}(T)$ vs. T across 100 evenly spaced values across the broad range between 0.01 to 2.0 generated by NumPy's `linspace` function. We found the minimum value to be on the lower end of our range from 0.01 to 0.2. We then generated 100 evenly spaced values across this range and found the T with the minimum $\text{NLL}(T)$ value to be approximately 0.09. This temperature value is then used to convert all similarity scores to percentages and to generate our graphs. For the RF model, we use the same methodology to format the input data that in turn allows the model to predict the color label.

To quantify the accuracy of the model in terms of percentages, we simply use the percentage of trees that accurately predict the color label. We check whether the RF raw percentages are similar to the actual probabilities by calculating the mean squared error between the average predicted probabilities and the empirical probabilities as follows:

$$\text{MSE} = \frac{1}{N_{\text{bins}}} \sum_{\text{bins}} (\bar{p}_{\text{bin}} - \bar{e}_{\text{bin}})^2 \quad (5)$$

where there are 10 bins (N_{bins}) that correspond to the 10 ranges, $[0,0.1)$, $[0.1,0.2)$, $\dots$, $[0.9,1.0]$. We have a total of 400 predicted probability values (5 colors, 4 lighting conditions, 4 readings, 5 color probabilities). We allocate each value to the bin it falls under. For each bin, the mean probability, $\bar{p}_{\text{bin}}$, is given by,

$$\bar{p}_{\text{bin}} = \frac{1}{n_{\text{pred}}} \sum_{i \in \text{bin}} p_i \quad (6)$$

where n_{pred} is the number of predictions in the bin and p_i is each predicted probability value. For each bin, the empirical accuracy $\bar{e}_{\text{range}}$, is given by,

$$\bar{e}_{\text{bin}} = \frac{1}{n_{\text{pred}}} \sum_{i \in \text{bin}} y_i \quad (7)$$

where y_i is the true label for prediction i . $y_i = 1$ if the prediction is accurate and $y_i = 0$ otherwise. According to our calculations the $\text{MSE} = 0.042$. Given the sample size of 400, this value indicates moderate to good forecasting accuracy

⁵A lower value of $\text{NLL}(T)$, the more accurate the calibrated probabilities are to empirical probabilities. For instance, given a hypothetical sample with the soft-maxed probabilities and true-color label of orange: [Red = 0.3, Orange = 0.6, Yellow = 0.05, Green = 0.03, and Blue = 0.02] the $\text{NLL}(T)$ value would be $-\log(0.6) \approx 0.5$. While a lower temperature value would produce hypothetical soft-maxed probabilities of: [Red = 0.07, Orange = 0.9, Yellow = 0.01, Green = 0.01, and Blue = 0.01] would have a lower $\text{NLL}(T)$ value $-\log(0.9) \approx 0.15$. This essentially penalizes when there is a lower calculated probability for the correct label. However if in the example the true-color label was instead red, a lower temperature value would result in a higher $\text{NLL}(T)$ value as $-\log(0.07) > -\log(0.3)$. This penalizes when there is a higher calculated probability for the wrong label. The temperature value that minimizes the $\text{NLL}(T)$ value will be the most accurate representation of the true empirical probabilities

according to common rule-of-thumb (see, Niculescu-Mizil and Caruana, 2005). The RF model’s probabilities are, therefore, reasonably calibrated and provide a good representation of the true probabilities.

Finally, from the calibrated predicted probabilities for both the Cosine Similarity (Cos Sim) system and the RF (ML) system, we analyze color classification performance graphically. We produce four graphs, one for each lighting condition (LED, WHITE NORMAL, WHITE DIM, and YELLOW). These graphs contain ten box plots, one for each color-system pair. We generate the box plots from four data points per lighting-condition–color pair in each system.

C. Findings

In this section, we summarize the main results that are depicted in the graphs and summarized in a table below.

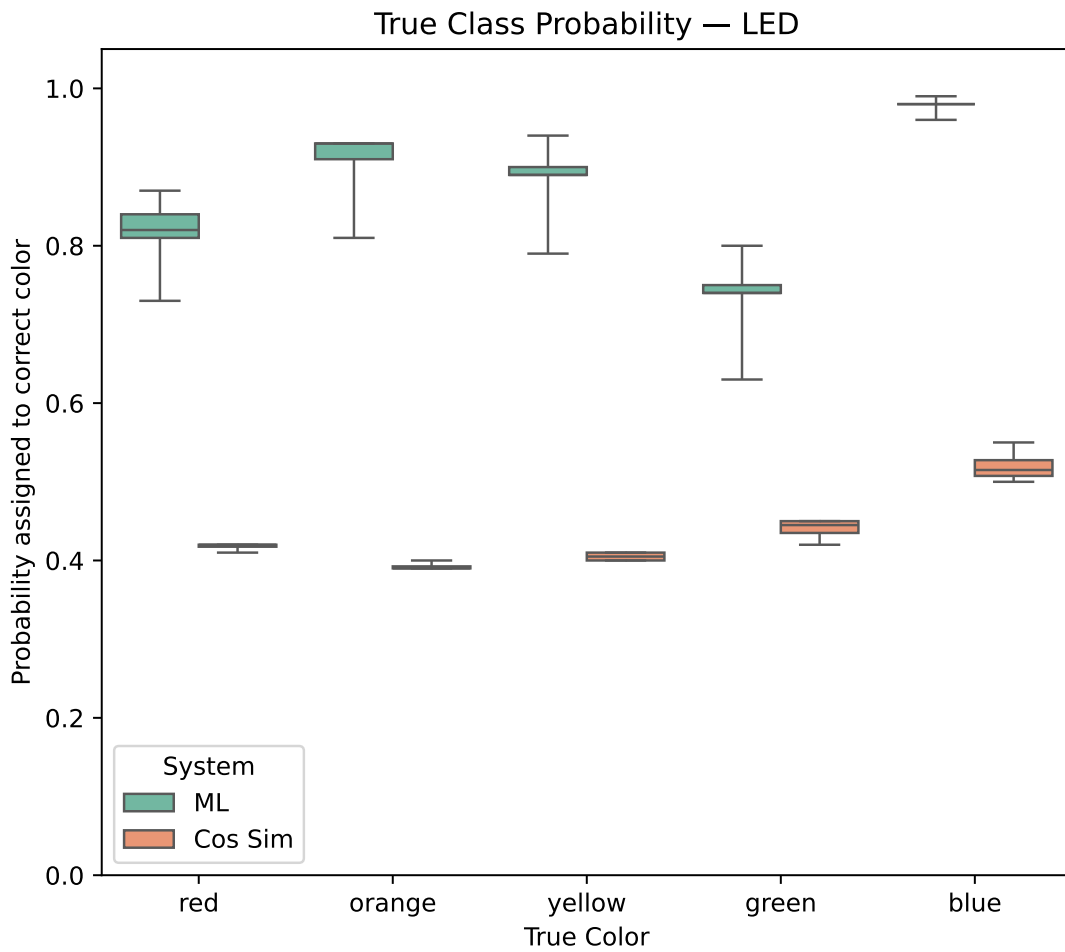


Fig. 1. Boxplots of predicted probabilities for each color and system under sensor LED illuminance.

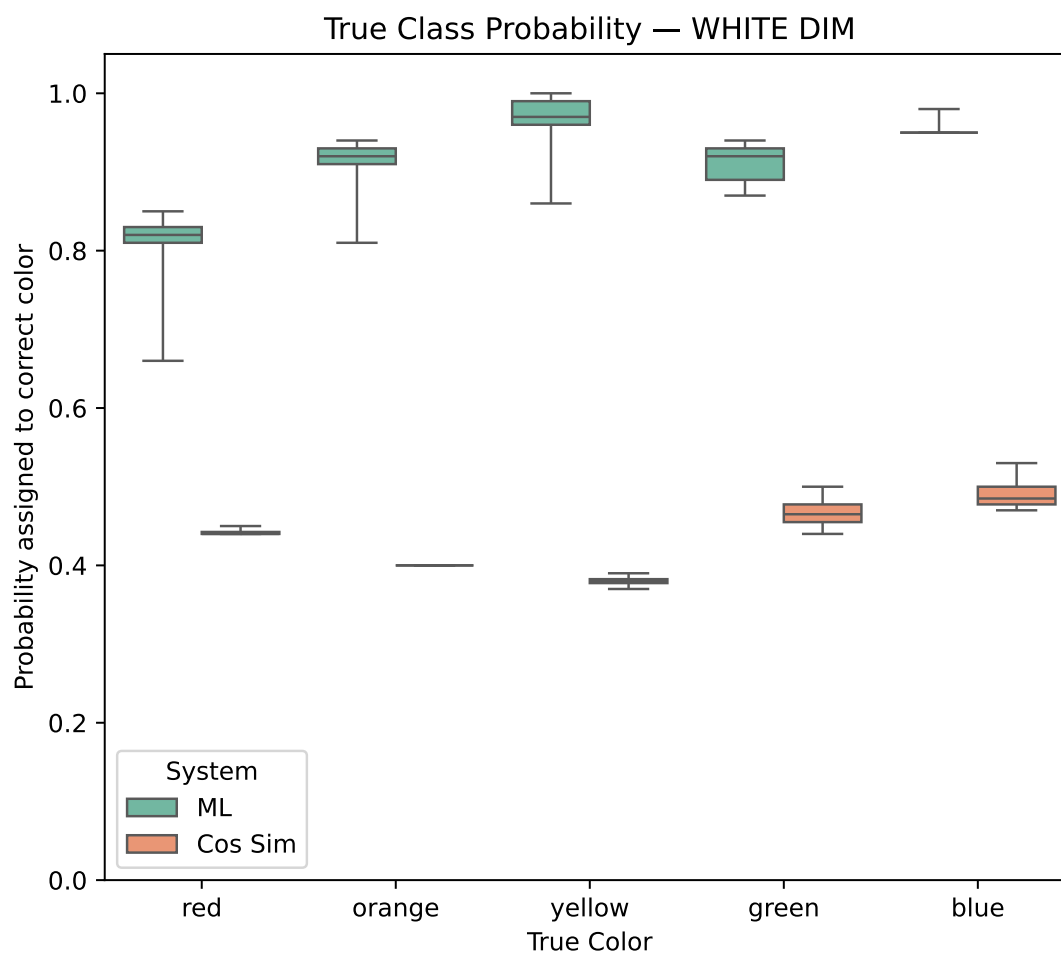


Fig. 2. Boxplots of predicted probabilities for each color and system under white illuminance.

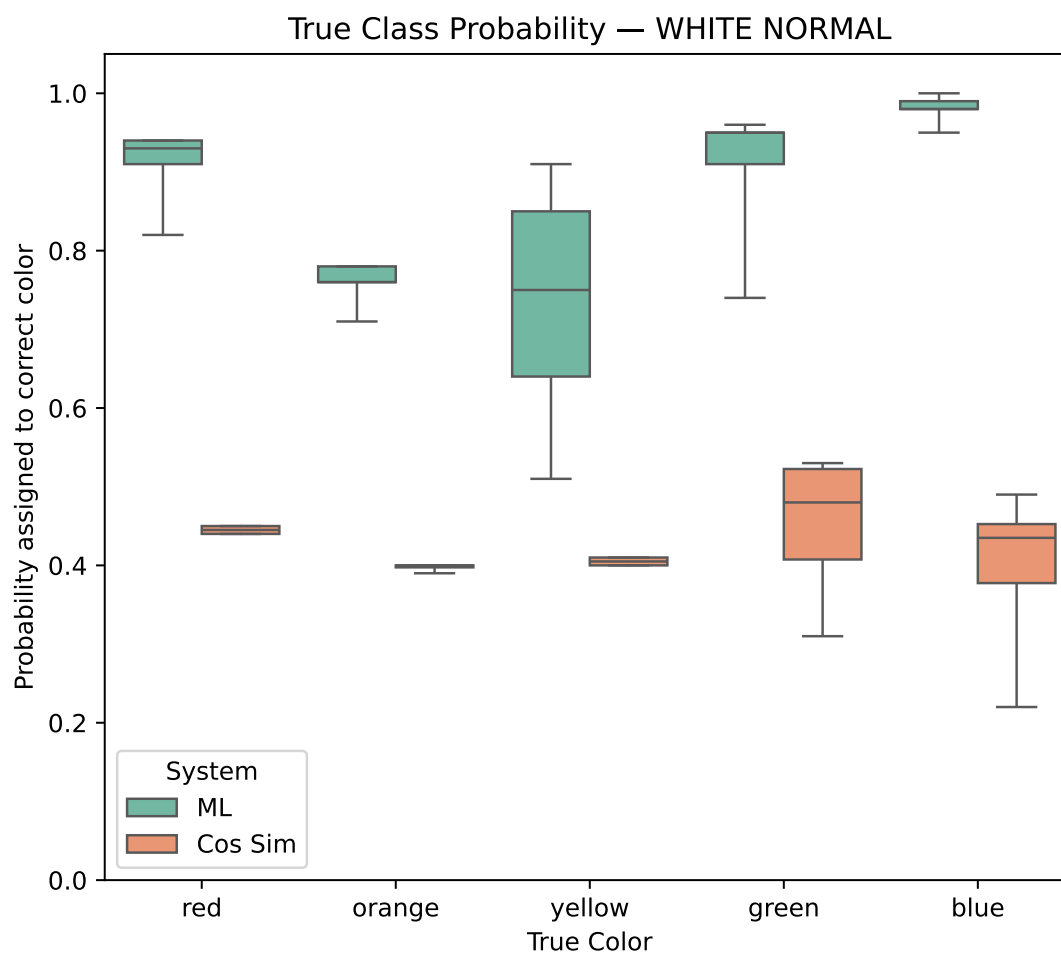


Fig. 3. Boxplots of predicted probabilities for each color and system under dim white illuminance.

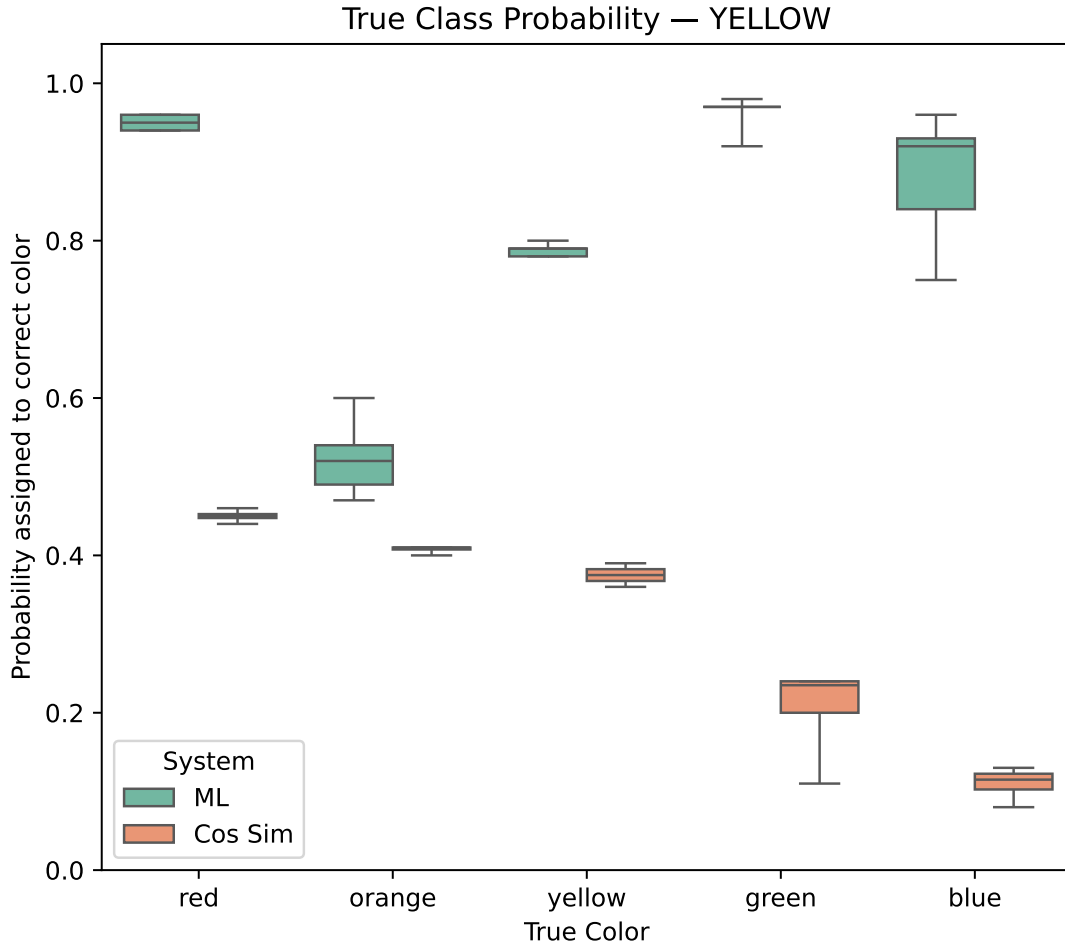


Fig. 4. Boxplots of predicted probabilities for each color and system under yellow illuminance.

Figures 1-4 correspond to the four lighting conditions that we use. The boxplots in all figures demonstrate that the spectral color detection with machine learning achieves a higher classification accuracy than the baseline color detection test. Specifically, the box plots consistently show higher probabilities for the ML system, indicating that the distributions produced by the training model are sharper. The inconclusive effects of yellow light are apparent in (Figure 4) where the predictions for the orange sample under the trained model and the predictions for green and blue under the baseline model are significantly less accurate.

Under LED lighting, the model correctly classified all five colors with high probability ($p > 0.9$ for the true class). Performance remained stable under dim white light, though there was a slight drop in confidence for yellow and orange samples. Under normal white light, classification accuracy stayed comparable to LED conditions. The most challenging scenario was yellow lighting, where confusion between yellow and orange slightly increased, as shown by overlapping probability distributions.

To evaluate the accuracy of the ML model relative to the Classical model, we computed the Root Mean Squared (RMS) error for each approach. The RMS error reflects the average magnitude of the deviation between the predicted probabilities and the ideal probability of 1 for the predicted class.

As displayed in Table I, the calculated RMS value across 100 predictions for the ML model was 0.1948 ± 0.0379 . The

calculated RMS value across 80 predictions for the Classical model was 0.5759 ± 0.3316 . There was an approximately 87% reduction in the percentage of mean squared error for the ML approach relative to the baseline approach.

TABLE I
FINDINGS SUMMARY (ACCURACY VALUES ARE $\bar{x} \pm \sigma$)

Lighting Condition	Classical Accuracy	ML Accuracy	RMS (Classical)	RMS (ML)
<i>Panel A: Model Comparison</i>				
LED Light	0.44 ± 0.05	0.84 ± 0.12	0.56	0.20
Normal White	0.43 ± 0.04	0.86 ± 0.13	0.57	0.19
Dim White	0.44 ± 0.05	0.90 ± 0.08	0.56	0.12
Yellow Light	0.40 ± 0.07	0.82 ± 0.17	0.60	0.24
<i>Panel B: Overall Performance</i>				
Average	0.43	0.86	0.57	0.19

To further check for robustness, we conducted a Welch's two-sample t -test to assess whether the margin of error was statistically significant. For two degrees of freedom, $t(93.5) = -4.37, p < 0.001$. Since the p -value of 3.3×10^{-5} indicates that the probability of observing this difference by chance is less than 0.0033%, there is strong statistical evidence that the ML model is 87% more accurate than the baseline model.

IV. CONCLUSION

In this project, we created a compact spectrometer-shroud system and utilized the 10 data channels from the AS7341 spectral sensor to detect colors under different ambiance. We simulated ambient light by creating four different controlled lighting conditions. To establish a reference, we first obtained calibrated values for each of the five colors under the four different lighting conditions. We then ran cosine similarity on these calibrated readings and our test readings. For machine learning, we trained a Random Forest model, utilizing the decision trees to produce direct probability values. Analyzing the results made it clear that our machine learning color detection test was more reliable and accurate than the baseline detection even under challenging lighting conditions such as white dim and yellow.

Our small-scale investigation can be extended to color detection tests under true uncontrolled ambient light. The ambiguity in the data for yellow light, in particular, could be further investigated in future work. To make this experimental test more reliable, one would need to determine the appropriate calibration methodology for raw data beyond a simple white reference.

In our experiments, each test trial was administered with the same sensor to object distance. Detection probabilities could vary with distance. It would be informative for future research to utilize more powerful sensors and determine whether our detection methods improve color detection accuracy at greater distances.

REFERENCES

- [1] Contreras, J. *et al.*, “Explainable artificial intelligence for spectroscopy data,” *Patterns*, 2024.
- [2] Erba, M., Buzzelli, M., and Schettini, R., “RGB color constancy using multispectral pixel information,” *Journal of the Optical Society of America A*, vol. 41, no. 2, pp. 185–195, 2024. doi:10.1364/JOSAA.506186.
- [3] Flachot, A., Lacherez, R., and Landy, M. S., “Deep Neural Models for Color Discrimination and Color Constancy,” *Journal of Vision*, vol. 22, no. 13, pp. 1–17, 2022. doi:10.1167/jov.22.4.17.
- [4] Gené-Mola, J., Sanz-Cortiella, R., Rosell-Polo, J. R., Morros, J. R., Company, J., Llorens, J., and Escolà, A., “Fruit detection and 3D location using instance segmentation neural networks and structure-from-motion photogrammetry,” *Computers and Electronics in Agriculture*, vol. 169, pp. 105165, 2020. doi:10.1016/j.compag.2019.105165.
- [5] Hamidisepehr, A., and McDonald, A., “Classifying reflectance targets under ambient light: evaluation of ambient-light compensation methods and classifier performance,” *Sensors (Basel)*, 2020.
- [6] Khanal, B. *et al.*, “Machine-learning-assisted analysis of colorimetric assays,” *ACS Omega*, 2021.
- [7] Koskinen, M., Hautalampi, A., Leppänen, L., Eerola, L., Kämäräinen, J.-K., Ahonen, T., and Kannala, J., “Single-pixel spectral color constancy,” *International Journal of Computer Vision*, vol. 131, no. 10, pp. 2522–2539, 2023. doi:10.1007/s11263-023-01867-x.
- [8] Niculescu-Mizil, A., and Caruana, R., “Predicting good probabilities with supervised learning,” *Proceedings of the 22nd International Conference on Machine Learning (ICML 2005)*, pp. 625–632, 2005.